

Analizador Léxico y Sintáctico

Proyecto 2



Manual_Tecnico

Universidad San Carlos de Guatemala

Facultad de ingeniería

Escuela de Ciencias y Sistemas

Introducción

Este manual técnico describe la arquitectura y el funcionamiento del compilador desarrollado en C++ y Qt para la lectura de scripts de tareas (.task). El documento detalla la implementación de un Autómata Finito Determinista (AFD) para el escaneo léxico y un Analizador Sintáctico Descendente Recursivo para validar la estructura del tablero. Se incluye la descripción de algoritmos, diagramas de clases y la lógica detrás de la generación de reportes dinámicos en HTML y la construcción del árbol de derivación en formato DOT. Esta guía constituye el sustento técnico para la validación y escalabilidad del sistema.

Diagrama de Componentes UML

Arquitectura del Sistema

El sistema implementa una arquitectura de Diseño por Capas, lo que permite una separación clara entre la interfaz de usuario, la lógica de procesamiento (Lexer y Parser) y la salida de datos.

Componentes Principales

Capa de Interfaz (GUI): Implementada en MainWindow. Gestiona la carga de archivos, captura de eventos y visualización de resultados en objetos QWidget.

Capa Léxica (Lexer): Implementada en LexicalAnalyzer. Contiene el motor de reconocimiento basado en un AFD, transformando el texto en una lista de objetos Token.

Capa Sintáctica (Parser): Implementada en SyntaxAnalyzer. Evalúa la lista de tokens contra la Gramática Libre de Contexto (GLC) y construye el árbol de derivación en código Graphviz.

Capa de Errores: Implementada en ErrorManager. Centraliza y clasifica las fallas (Léxicas o Sintácticas) y su nivel de gravedad.

Capa de Reportería: Implementada en ReportGenerator. Exporta las estructuras analizadas a reportes HTML interactivos (Kanban y Carga por Responsable).

Diagrama UML

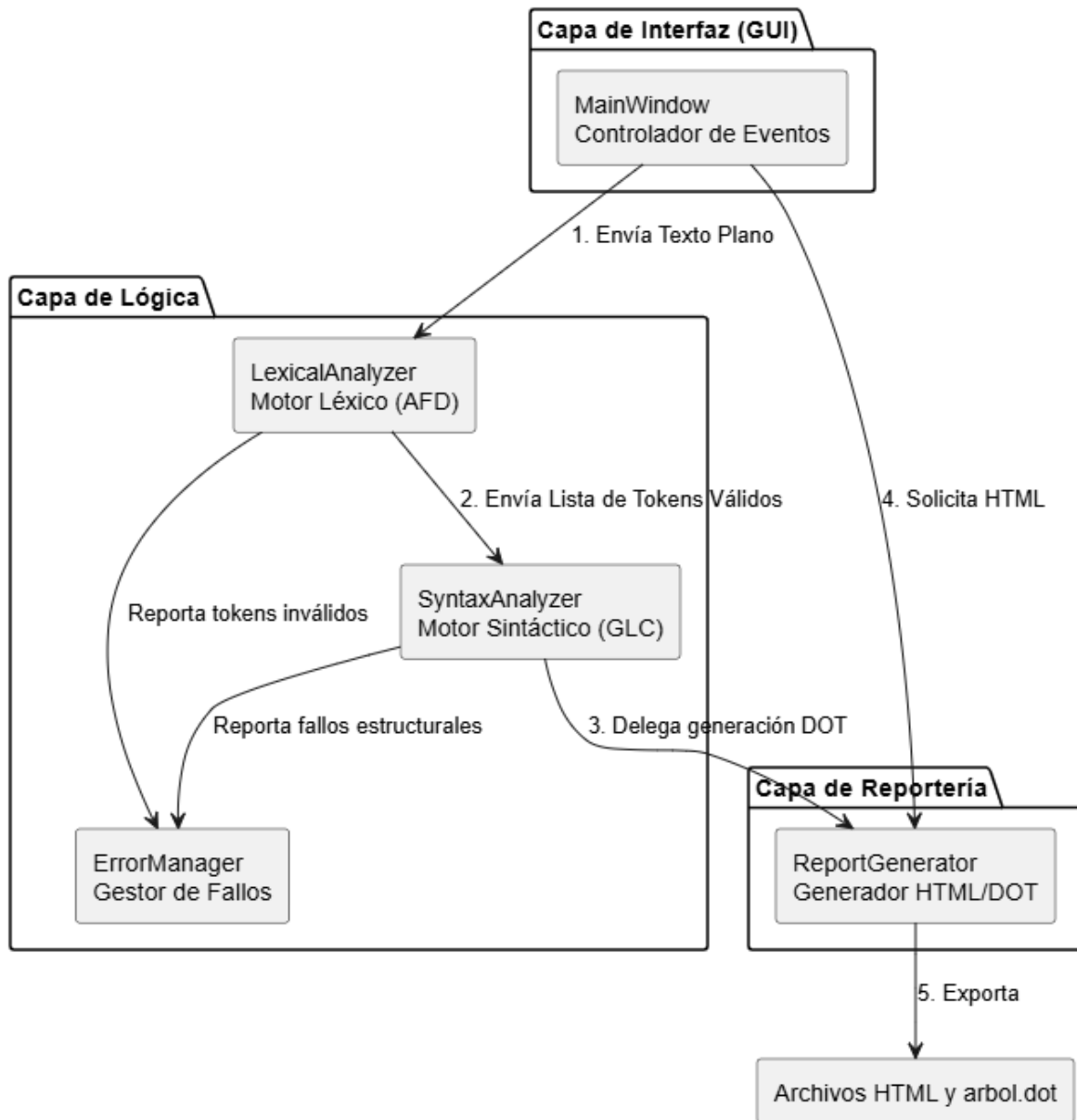


Diagrama de Clases

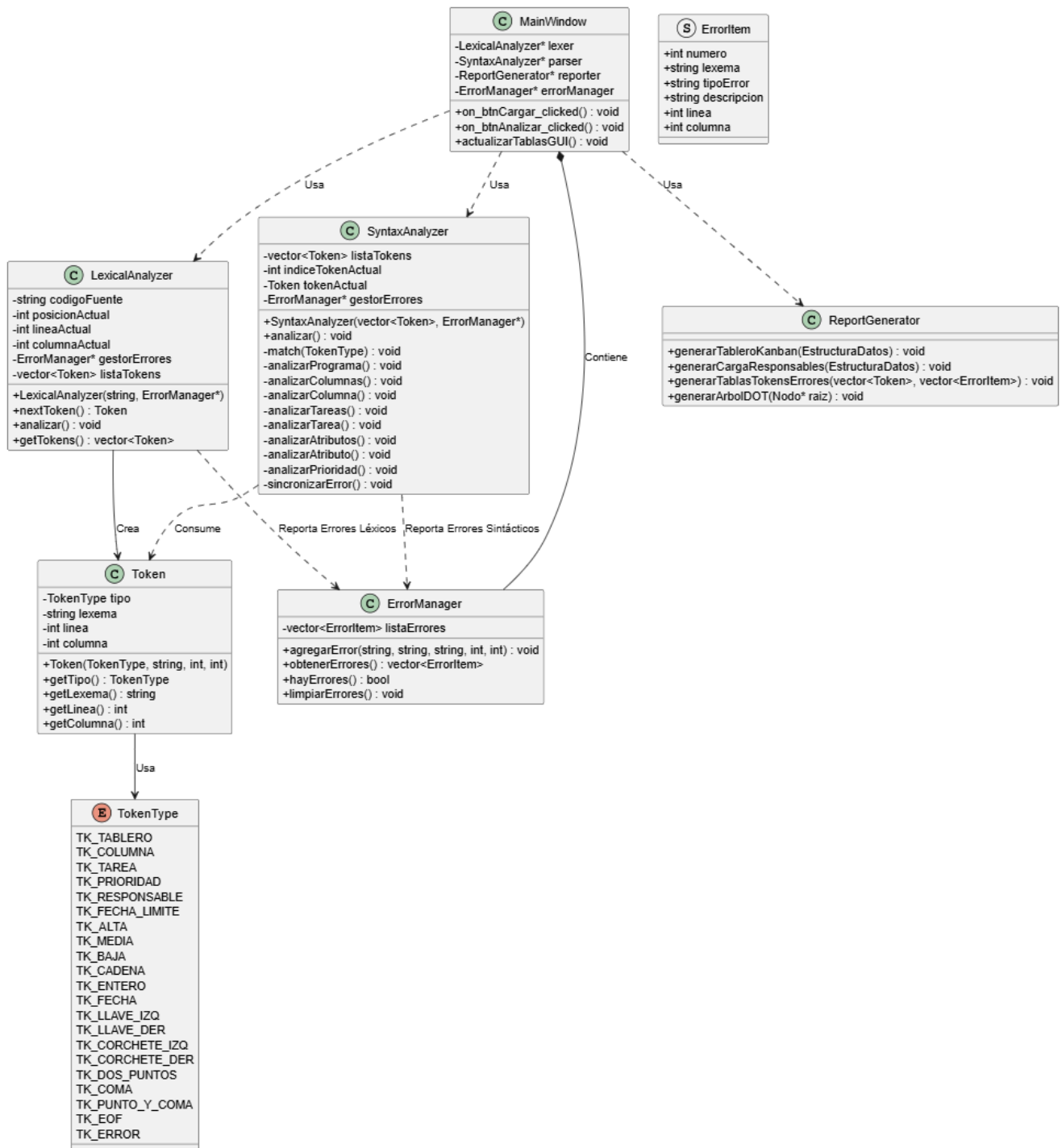
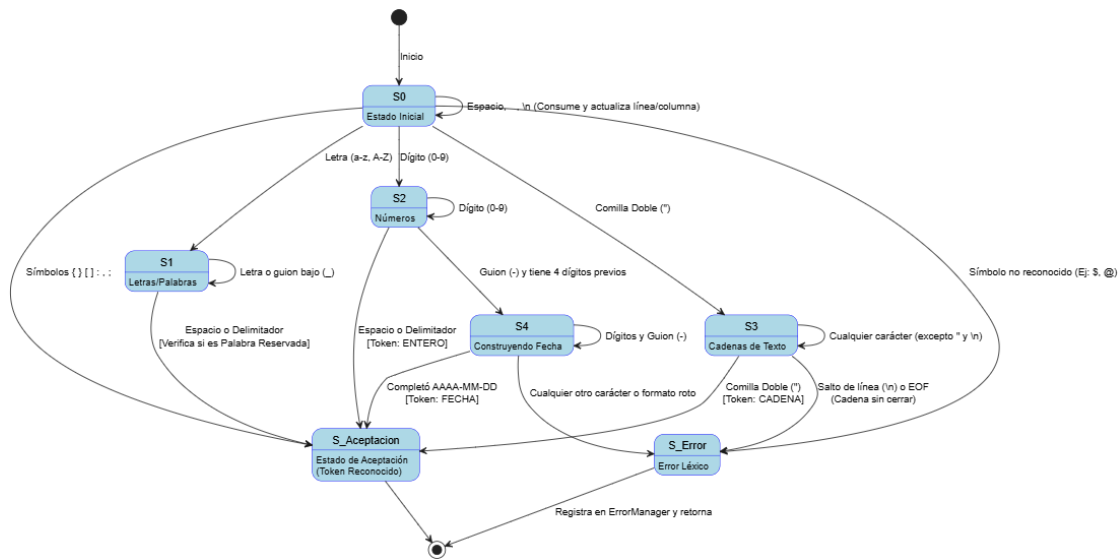


Diagrama de Autómata Finito Determinista AFD



Algoritmo de Tokenización (Escaneo Lineal)

El corazón de la fase léxica es un algoritmo de un solo paso (Single-pass) implementado en la clase `LexicalAnalyzer`. Este método recorre el código fuente de inicio a fin sin retroceder, garantizando una complejidad temporal y eficiencia de $O(n)$.

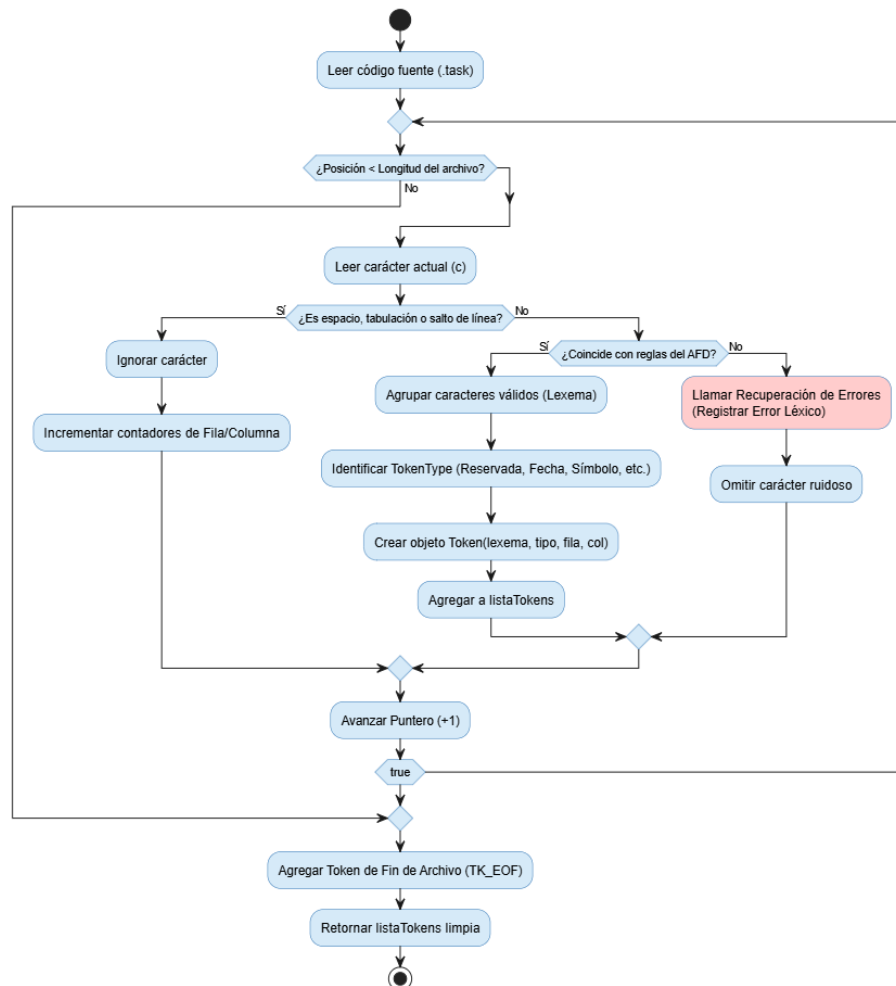
Lectura de Caracteres: Se utiliza un puntero de posición global (`indiceActual`) que itera sobre un objeto `std::string` que contiene todo el código del archivo `.task`.

Manejo de Espacios y Saltos: El algoritmo ignora activamente espacios en blanco (), tabulaciones (`\t`) y retornos de carro (`\r`). Al detectar un salto de línea (`\n`), incrementa el contador global de filas y reinicia el de columnas, asegurando una trazabilidad exacta para el reporte de errores.

Evaluación de Estados: Por cada carácter válido, el programa evalúa las condiciones del AFD mediante funciones de validación como `isAlpha()` e `isDigit()`. Si es una letra, entra al ciclo de formación de palabras para distinguir entre identificadores o palabras reservadas (ej. `TABLERO`, `ALTA`). Si detecta un patrón numérico con guiones, lo evalúa como una posible fecha (`YYYY-MM-DD`).

Generación de Objetos: Al concretar un estado de aceptación (lexema válido), la subcadena extraída se encapsula en un objeto de la clase `Token`, guardando su tipo (`TokenType`), valor, fila y columna de origen, para finalmente insertarlo en la `listaTokens`.

Diagrama de Algoritmos de Tokens



Algoritmos de Análisis (Single-pass y Descenso Recursivo)

El núcleo léxico utiliza un escaneo lineal que recorre el texto sin retroceder. Por su parte, el analizador sintáctico utiliza un algoritmo de Descenso Recursivo, donde cada regla de producción gramatical (ej. <tarefas>, <columna>) se implementa como una función en C++ que se llama a sí misma.

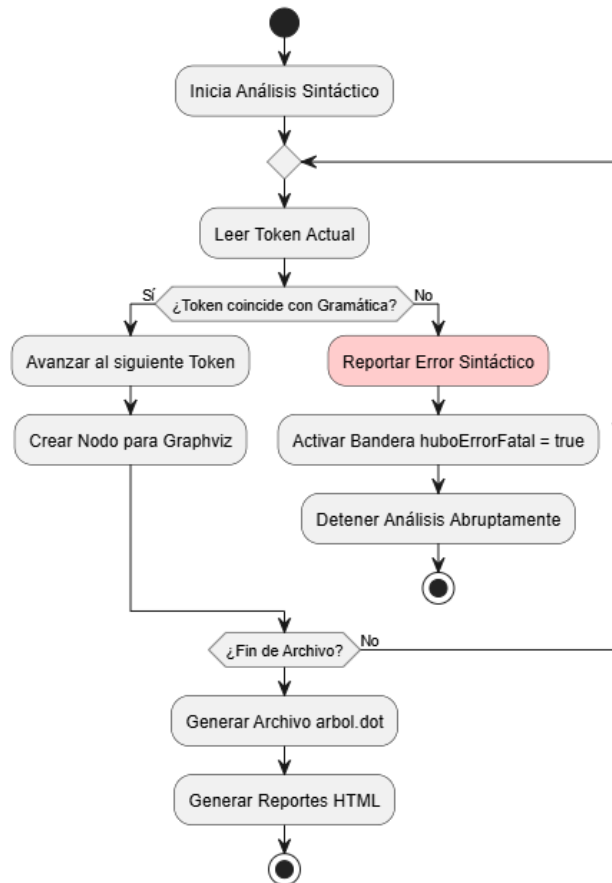
Manejo Estricto de Errores

Se establecieron dos políticas distintas de recuperación:

Errores Léxicos (No detentivo): Si un carácter no pertenece al alfabeto (ej. \$, @), se guarda como un error en el gestor, se omite y el autómata continúa limpiando el archivo.

Errores Sintácticos (Fatal): Si la estructura de tokens rompe la gramática esperada, el analizador registra el fallo y detiene la ejecución inmediatamente para evitar la generación de diagramas corruptos.

Diagrama de Algoritmos Errores



Decisiones de Diseño

Lenguaje C++ y Framework Qt: Se seleccionó por su rendimiento óptimo y el control manual sobre las estructuras de datos sin depender de expresiones regulares, cumpliendo con el enfoque pedagógico de construir compiladores desde cero.

Salida en Formato DOT (Graphviz): Se optó por generar código .dot para el árbol de derivación debido a su precisión algorítmica para organizar jerarquías visuales complejas, asignando colores específicos a los nodos terminales (celeste) y no terminales (azul oscuro) mediante sus etiquetas de configuración.

Tolerancia y Restricción de Errores: La decisión de separar el comportamiento ante fallos mejora la fiabilidad. Ignorar la basura léxica permite extraer todos los tokens sanos, mientras que abortar ante fallos sintácticos asegura que los tableros Kanban y reportes de carga solo se rendericen si el archivo es estructuralmente perfecto, protegiendo la integridad de la base de datos visual.

Conclusiones

Eficacia del Modelo Híbrido: La combinación de un Autómata Finito Determinista (AFD) para el análisis léxico y un Analizador Sintáctico Descendente Recursivo demostró ser una metodología altamente eficiente para procesar el lenguaje TaskScript. Esta estructura permite una clasificación precisa de lexemas y una validación jerárquica de la gramática con una complejidad temporal óptima de $O(n)$.

Gestión Integral de Errores: La implementación de una estrategia diferenciada ante fallos garantiza la estabilidad del sistema. Mientras que la recuperación léxica permite al usuario corregir múltiples símbolos inválidos en una sola pasada, la detención fatal ante errores sintácticos asegura la integridad de los reportes generados, cumpliendo estrictamente con las normativas de validación técnica.

Valor de la Visualización de Datos: La integración de herramientas de visualización como Graphviz (DOT) y reportes dinámicos en HTML5 transforma scripts de texto plano en información estructurada y comprensible. Esto facilita la auditoría del proceso de compilación mediante el árbol de derivación y optimiza la gestión de proyectos al presentar tableros Kanban y estadísticas de carga de forma visual.